CPU LLM: Evidence-Based AI Engine

Answer questions from your documents and explicit rules, with source evidence and CPU-first execution.

Implementation whitepaper and developer guide

Product: cpu-llm 1.10.0 **Author:** Amin Parva, Independent Engineer **Date:** November 2, 2026 **Edition:** 1.0, based on the published 1.10.0 implementation **Audience:** application developers, technical evaluators and engineering leads

This paper explains how to install cpu-llm, ingest records, select its execution paths, interpret results and integrate its Python and HTTP interfaces. It documents implemented behavior and engineering evidence. It is not a peer-reviewed clinical study, security certification or guarantee of factual correctness. Source code and executable examples remain the implementation authority. References at the end link to the version, contracts and saved runs.

1 Executive overview

cpu-llm is an Apache-2.0 Python library for CPU-first document question answering and explicit rule checks. It combines a trained retrieval projection, a pretrained sentence encoder, PostgreSQL/pgvector, symbolic source connections, conditional graph stages and bounded response construction. Applications embed the library or place its authenticated HTTP entry point behind their own service infrastructure. It can support self-hosted document search and evidence-backed question answering without requiring an unrestricted generative decoder for every answer.

The central integration decision is which response path a request needs. Strict evidence mode returns recognized source quotations, explicit evidence chains, bounded exact calculations or nonanswers. The ordinary graph supports additional producers such as supplied-field finance checks, simple event frames, summaries and layouts. An optional tone node constructs a verified companion from a small accepted-fact subset. A reviewed built-in pack handles basic social requests separately from document knowledge.

These paths have different contracts. A quotation is not a verification of its source. A calculation is not a clinical recommendation. A generated companion is not a verbatim quotation. A social response is not retrieved evidence. Applications should preserve those distinctions in their interface and logs.

The name does not imply that cpu-llm is a general replacement for ChatGPT or every RAG system. It targets inspectable, bounded document and rule workflows. The published release does not establish 100 percent accuracy, clinical safety, universal contradiction detection, or superiority over hosted LLMs.

2 Suitable workflows and boundaries

Document assistance. Retrieve supported facts from manuals, policies and records; show the original supporting text; ask for missing intent or retain a nonanswer when the contract cannot be satisfied. Documents must be ingested before the tenant can answer questions about them.

Healthcare and pharmacy demonstrations. The included fictional records exercise label strengths, ingredients, storage, recorded diagnoses and allergies, conflicting labels, missing information and refusal of unsupported treatment questions. A recorded-diagnosis lookup does not diagnose from symptoms. An arithmetic comparison with a supplied limit does not decide a safe dose. Authoritative sources, intended-use validation and independent

clinical review are still required for any clinical decision-support claim.

Financial review support. RefundProof, LedgerCovenant and Ledgerlock validate recognized caller-supplied fields and policies. They can make numeric inputs, decision traces and refusals inspectable. Source-authority labels in the request are inputs, not independent authentication of financial documents. These checks are not tax/legal advice or automatic authority to recover money or declare a default. The library's components are not the same deployment as the website's separate hosted financial products.

Application conversation. Greetings, identity, wellbeing, capabilities, thanks, farewells and limitation questions receive original reviewed responses. This improves basic interaction without importing another assistant's identity or treating a dialogue corpus as medical knowledge. It does not supply arbitrary world knowledge or unrestricted conversational reasoning.

3 Install and run the packaged examples

Python 3.11 or newer is required. The release validation used Windows and Python 3.12.10; it did not freshly execute every supported operating system/version.

```
python -m pip install "cpu-llm==1.10.0"
python -m pip check
cpu-llm-demo --help
```

Configure PostgreSQL with pgvector and set `VECTORPRISM_PG_DSN` to the appropriate database. The wheel contains the trained VectorPrism projection and demo data. The pretrained `sentence-transformers/all-mpnet-base-v2` encoder assets must be downloaded on first use or cached beforehand. Installation does not start the database or ingest a corpus. First startup includes asset loading; there is no universal startup, throughput, memory or training-time guarantee.

```
cpu-llm-demo healthcare --out healthcare-demo-new-run
cpu-llm-demo conversation --out conversation-demo-new-run
```

Each output directory must be new. These commands create isolated namespaces, ingest packaged fictional records and save actual responses and expected-result checks. They work from an installed package without Git or repository paths. No model retraining is needed. Once model assets are cached, offline settings can be used; dependency provisioning and deployment egress controls remain the operator's responsibility.

```
HF_HUB_OFFLINE=1
TRANSFORMERS_OFFLINE=1
```

The lines above name environment settings, not portable shell assignment syntax. For example, PowerShell uses `$env:HF_HUB_OFFLINE = '1'`. Offline operation requires the assets to exist already. Optional spaCy parsing uses the `prompt_nlp` extra and a separately installed `en_core_web_sm` model. Coreferee, FastCoref and optional decoder checkpoints also require their own compatible assets. They are not bundled in the small wheel.

4 Ingestion and the first Python integration

Construct an appliance inside a guarded `main()` on Windows because its worker pool starts child processes. Reuse an appliance and close it with a context manager. Construction loads the retrieval stack and connects to storage; it is not a lightweight stateless function call.

```
from cpu_llm.documents import SourceDocument
from cpu_llm.pipeline import Appliance
from cpu_llm.access import CallerContext
```

```
def main():
    with Appliance(namespace="paper_demo", strict_evidence=True,
                   prompt_backend="rules") as app:
        receipt = app.ingest(SourceDocument("manual", (
            "The material of Example Device is steel.",
        )))
        caller = CallerContext.make(
            app.tenant_id, "reader", ["manual-p"]
        )
        result = app.ask(
            "What is the material of Example Device?",
            caller=caller,
        )
        print(receipt["ids"])
        print(result["answer_status"])
        print(result["reply"]["text"])
        print(result["answer_evidence"])

if __name__ == "__main__":
    main()
```

The source name produces paragraph IDs such as `manual-p0001`. Caller scopes can authorize an exact document ID or a source prefix such as `manual-p`. Source boundaries are explicit; arbitrary string-prefix similarity is not an authorization rule. Files and supplied paragraphs can also carry structured connections into the source-backed graph.

The appliance supports structured `SourceDocument` inputs and recognized file formats, including text/Markdown, HTML, PDF, Word, spreadsheets and EPUB. This does not establish support for scanned-page OCR, every table layout or arbitrary clinical document interpretation. Evaluate the actual reader output before using it as a source. Reingestion updates the source's stored paragraphs and connections; it is not retraining the encoder or a general sentence generator.

Named namespaces map to normalized tenant/schema names and cache folders. Different raw names can normalize to the same slug. Provision canonical tenant identities rather than accepting arbitrary names as proof of isolation. Use a stable, deliberately separated working directory/cache for a deployment and database combination; database state and graph/cache state are distinct.

5 Execution paths and the six retrieval channels

The request boundary checks caller scope and guards raw input. With small talk enabled, complete reviewed social phrases use a built-in response. A leading greeting before a substantive request can be removed with an explicit provenance record; the remaining request is retained. Strict mode then follows its separate evidence path. Ordinary requests proceed through planning and the graph.

The ordinary graph's core progression is:

```
guard -> retrieve -> resonate -> six channel readings
      -> catalog -> detail -> spans -> think -> cognize
      -> grammar -> shine -> requested optional stages
      -> model01 -> reply -> optional discover
```

Conditional branches and delivery holds matter; this diagram is not a promise that every optional stage runs. Final response judgment/repair is part of the public request pipeline. The optional stylistic output subgraph and final layout rendering are separate from the core graph.

The six channels are **dense**, **relational**, **disentangled**, **hyperbolic/taxonomy**, **identity**, and **causal/time**. They provide retrieval and relation signals, not independent proof of a claim. The shipped projection enables dense and

identity heads; symbolic connections in other channels do not mean all six projection heads are trained and enabled. Epistemic/source support is separate metadata. Connections are stored alongside vectors rather than recoverable as a complete word network from vector coordinates alone.

Strict lookup uses a scoped SQL snapshot and recognized literal source forms; its conflict check is not a vector top-k vote. Curated tone features and the ten output tone preferences are a different system from these six retrieval channels. Adding a channel score or tone weight never creates a missing fact.

6 Node reference and request controls

Developers normally use `ask` and its options, not direct calls to internal graph nodes. The implementation registers 27 core nodes: 21 named stages plus six channel-reading nodes. The inventory below describes responsibility, not a standalone public API guarantee for each internal function.

Core and interpretation stages

- `guard` checks the request's allow/block rules. Raw input is also guarded at the request boundary before early-return paths.

- `retrieve` obtains candidates within the request's source/time constraints.

- `resonate` supplies retrieval/resonance readings; a score is not source truth.

- `dense`, `relational`, `disentangled`, `hyperbolic`, `identity`, `causal` evaluate the six channel signals and disclose head enablement.

- `catalog` associates available stored proposition/catalog information.

- `detail` expands supported detail within the response plan.

- `spans` prepares source-bound spans and value/grammar constraints.

- `think` runs configured Thinker evaluation/context, including caller-supplied causal/measurement inputs when present; it does not supply clinical authority.

- `cognize` runs the configured cognition stage and its available signals.

- `grammar` applies the bounded response construction/grammar checks.

- `shine` contributes an evidence-aware delivery gate. A passed component check is not verification of the real-world truth of a source.

Conditional stages and delivery

- `deep` widens authorized retrieval when explicitly requested; it is not an automatic web-search agent.

- `gather` collects supported article/context material before `imagine`.

- `imagine` handles the bounded imagination path; the release does not offer unrestricted creative generation from missing facts.

- `compare` handles supported explicitly named comparisons.

- `brief` produces supported summary/list/explanation units within budgets.

- `math` evaluates recognized supplied arithmetic/comparison inputs.

- `finance` runs selected structured financial checks.

- `frame` handles supported simple event-role questions from stored evidence.

- `model01` constructs bounded sentences. Its training interface surveys signals; it does not train and write a general neural generation checkpoint.

- `reply` composes the canonical response for subsequent delivery judgment.

- `discover` adds requested follow-up discovery suggestions after reply.

The pre-graph `conversation.reply` and `strict_evidence.answer` paths are outside that 27-node graph. The opt-in output stages `stylistic -> style_verify` are also separate. Not all stages are automatically invoked for every question.

`deep`, `math`, `finance` and `discover` have explicit Python switches. Recognized prompt forms can request comparisons, summaries, lists, sources, explanations and frames. A long-response request adjusts retrieval/delivery budgets; it does not automatically request the deep-search node. Presentation flags select built-in tables, bullets, numbered lists or reports after judgment. They do not permit missing claims to be invented to fill a length target.

The flag registry maps each supported flag to its consumer and enforces optional node routing. Advanced payment, authority, objective, hypothesis and causal-graph inputs require the corresponding typed contracts; a keyword alone is not a validated business or clinical decision procedure.

7 Strict evidence mode

Enable per request with `strict_evidence=True`, with the strict HTTP header/body flag, or with a standalone first-line `[strict-evidence]` prompt control. A constructor or service strict floor cannot be disabled by a weaker request. The ordinary `realization="deterministic"` setting by itself is not a reliable disable switch for the separate automatic fact-bridge fallback. Strict mode bypasses that learned path and the ordinary generation/specialist nodes.

Supported strict forms include a single `What is the <attribute> of <entity>?`, up to eight bounded questions, explicit attribute lists and up to 32 recognized direct attributes in an overview. A one-hop join requires an explicit membership and an explicit universal rule; a class average does not license an individual fact. A complete ambiguous/missing task is retained as a nonanswer rather than silently removed from a batch.

Recognized source forms include canonical attribute statements, possessive statements, `has` constructions and `for` variants. Whole chunks remain atomic so a qualifier in a following sentence is not silently discarded. Recognized negation, uncertainty and conflicting values withhold the slot. Other prose contradictions may be missed. Exact scalar mass, volume and time equivalence is supported for the specified unit symbols; arbitrary dose conversions, ranges, compound units and clinical equivalence are not inferred.

One SQL snapshot considers at most 10,000 authorized time-valid chunks and eight million text characters. Overflow refuses to select a factual answer. `as_of` selects stored historical validity, not proof that a statement remains medically current outside the database. `scan_complete` describes the bounded database scan, not completeness of medical knowledge or semantic contradiction coverage.

Strict arithmetic supports bounded explicit addition, subtraction and multiplication, plus recognized same-unit greater-than comparisons. A scalar source value can be multiplied only after unambiguous supported retrieval. Conflicting or uncertain source inputs do not produce a calculation. The result records operands/operator/value/unit and is labeled arithmetic, not treatment advice. There is no arbitrary expression

execution or clinical dosing policy.

Strict requests are stateless across turns. A unique earlier named entity in the same bounded batch can resolve `of it`; general cross-turn reference resolution is not enabled. Version 3 additionally types approved social turns as `builtin_conversation`, with empty document evidence and no clinical audit. Disable `small_talk` if a deployment requires only factual envelopes.

8 Consume results as contracts

`reply.text` is display text, not a universal truth/correctness certificate. Read the outcome, evidence, per-task results and producer-specific fields.

- `evidence_quote` supplies recognized literal source evidence or quoted join premises. In strict mode `answer` is null and `audit_passed` is false.

- `derived_calculation` supplies bounded deterministic arithmetic, with optional source proofs. It is not automatically a safe financial or clinical action.

- `conflicting` retains disagreement without selecting a value.

- `insufficient_evidence`, `unsupported` and `blocked` are nonanswers.

- `partial` means some requested tasks succeeded. Inspect every `task_results` entry; do not present a partial result as a complete answer.

- `completed` can describe mixed successful strict quotation/calculation tasks.

- `conversation` is a built-in social/policy response with no document proof.

- The ordinary path can also use `answered`, `status=needs_clarification` and producer/judgment fields. These envelopes are not all structurally identical.

Strict proofs contain document ID, literal quote, character offsets and a SHA-256 hash of the complete stored chunk. Offsets use Python string positions with an exclusive end, not UTF-8 byte positions. Ordinary evidence can instead use source `span` records. Preserve those distinctions. An unkeyed hash is a content fingerprint, not a publisher signature, clinical endorsement or durable audit log. Retain the response and matching source version in the application's own audit process when required.

Do not retry a held medical question by stripping safeguards or enabling a learned fallback. A quotation can be accurate to an incorrect source. A source scope check can succeed while the underlying document is obsolete. Applications own source quality and the authorized decision workflow.

9 Tone and bounded sentence construction

The Python `style` option requests an additional verified companion after the ordinary canonical answer has been judged. It supports color, location, shape, material and explicit weight through ten reviewed constructions. It does not retrieve more facts, train a new language model, or rewrite arbitrary specialist results. Generic clinical attributes and financial verdicts are outside its current adapter. Strict evidence mode bypasses it entirely.

The node snapshots an accepted fact frame and its proof. Relation/vocabulary and relationship-risk restrictions filter eligible constructions. Read-only curated features rank remaining choices by semantic fit, intensity and tone affinity, with a genericness penalty. Complete same-subject facts can be joined with additive `and`. An inverse-grammar verifier checks licensed clauses, claim coverage, budgets and the proof union before publication.

```
from cpu_llm.stylistic import StyleOptions, Relationship

# Inside an ordinary appliance after ingesting the relevant facts:
result = app.ask(
    "Tell me the color, habitat, shape, material, and weight of the Aster.",
    style=StyleOptions(
        tone="friendly", relationship=Relationship.COLLEAGUE,
        max_sentences=3, clauses_per_sentence=2,
    ),
)
side = result.get("stylistic", {})
if side.get("status") in {"generated", "unchanged"}:
    print(side["text"])
    print(side["proof"])
```

The actual fictional-object demonstration produced:

The Aster is blue, and you can find the Aster in coastal waters. The Aster has a round shape, and the Aster is made of steel. The Aster weighs 12.5 pounds.

`generated` and `unchanged` contain verified construction output. `skipped` and `rejected` publish no companion; `children` can expose child results without a safe aggregate. Canonical text and evidence remain unchanged. Label a companion as a derived paraphrase, not a source quotation or inherited audit of world truth.

Ten preference controls do not imply ten mature expressive styles. Poetic and other unsupported diction can return `neutral_fallback`. Requested tones are explicit and per-request, not automatically inferred or remembered. The default conversation pack uses separate reviewed social variants; it is not the factual style subgraph. The HTTP ask interface does not expose `style`.

10 Financial component example

The finance component can be called directly without constructing an appliance. Inside an ordinary appliance, the same structured prompt can use `app.ask(request, finance=True)`. A strict appliance does not enable finance specialists for one request; choose the direct component or a separate ordinary appliance for structured checks.

```
from cpu_llm.finance import validate_message

request = """Decide this LedgerCovenant test.
comparator: lte
threshold: 4.50
numerator: compliance_certificate 9000001
denominator: compliance_certificate 2000000
ratio places: 6
"""
result = validate_message(request)
print(result["text"])
print(result["checks"])
```

The stored six-place result is `4.500001`, which breaches `lte 4.500000` under the component's stated rounding policy. Presentation retains enough digits to explain that decision. A requested/default smaller display precision is a minimum, not permission to hide a deciding digit. Decimal strings and exact scaled values avoid accepting floating-point inputs as exact financial amounts. The scaled decision policy itself is bounded; this is not arbitrary-precision legal interpretation or authentication of a compliance certificate.

11 HTTP service and authorization

The service binds to `127.0.0.1:8765` by default. Prepare a credential file containing SHA-256 digests, canonical tenant IDs and permitted source prefixes. `cpu_llm.entry.secret_sha256(secret)` computes the digest; do not put the secret itself in that file or a public example. Send the secret using the Bearer header.

```
cpu-llm-serve --secrets credentials.json --strict-evidence
```

Use `--no-small-talk` for evidence-only envelopes. Path ingestion is off unless an ingest root is explicitly configured. The four HTTP routes use POST:

- `/v1/ingest`: `name` plus a `paragraphs` array, or a permitted server-side `source` path inside the configured root.

- `/v1/ask`: `message` plus supported fields listed below.

- `/v1/correct`: `document_id`, `prior_sentence`, `new_sentence`, `actor_id` and an explicit integer `timestamp` for versioned correction.

- `/v1/delete`: the authorized `source` to delete.

```
POST /v1/ask HTTP/1.1
Authorization: Bearer YOUR_LOCAL_SECRET
Content-Type: application/json
X-CPU-LLM-Evidence-Mode: strict

{"message":"What is the material of Example Device?"}
```

HTTP ask accepts `discover`, `imagine`, `deep`, `math`, `finance`, `as_of`, `session_id`, `strict_evidence` and `small_talk`. It does not expose every Python argument. Caller identity/source scope comes from the authenticated credential, not a caller object supplied in the request body. Strict header/body/operator activation uses enable-wins precedence; an operator small-talk disable cannot be re-enabled by a request. Invalid boolean or strict-header values are rejected.

`Entry` wraps reads and mutations with credential/source checks and restricts server-path ingestion to its resolved root. Direct `Appliance.ingest` and `delete_source` methods are trusted backend APIs, not authorization endpoints. Do not expose them directly to untrusted clients. The stdlib server is not a full production gateway: deploy TLS, access policy, request limits, logging and operational controls appropriate to the application.

12 Python API reference

The complete extracted signatures and registered-node inventory accompany this paper in `cpu-llm-1.10.0-api-reference.json`. The concise reference below covers the supported consumer entry points; internal node functions are not standalone public APIs.

Appliance construction: `checkpoint=None` and keyword-only `dsn=None`, `child_safety=None`, `namespace=None`, `on_escalate=None`, `reference_backend=None`, `prompt_backend=None`, `resonance_tags=False`, `strict_evidence=False`, `small_talk=True`. The reference backend defaults to disabled. Prompt `auto` uses available optional parser assets or a reported rules fallback; `rules` avoids the parser and `spacy` requires its assets. Resonance tags are off by default.

Ask options: `message`, optional `propositions`, then keyword-only `discover`, `imagine`, `deep`, `math`, `finance`, `payment`, `authority`, `objective`, `hypothesis`, `causal_graph`, `realization`, `account`, `grade`, `child_safety`, `session_id`, `caller`, `as_of`, `style`, `strict_evidence`, `small_talk`. Boolean

agent switches default false. Most optional typed inputs default null. `realization` defaults to `deterministic`, `small_talk` to the appliance policy through null, and `strict_evidence` to false subject to the constructor floor. Payment requires both proposal and authority snapshot. Treat advanced typed inputs according to their component contracts; their mere presence does not establish independent factual verification. Accepted realization labels are `deterministic`, `evidence_neural` and `model01`; these are not three trained decoder checkpoints. In the current single-turn implementation the request-level `grade` and `child_safety` arguments are discarded. Their presence in the signature must not be marketed as effective per-request safety enforcement or clinical policy.

Data and lifecycle: `ingest(source)` returns an ingestion receipt; `index_chunks` stores caller-split paragraphs; `catalog_chunks` additionally registers repeatable catalog/proposition information; `upsert(id, text)` is a single-chunk helper. `correct` accepts a document ID, exact old/new sentence, actor, timestamp and optional caller. `delete_source(source)` removes the named source. `clear_session_state()` clears conversation/planning state; `close()` releases workers/resources. Use `Entry` for untrusted mutations.

Concurrent requests: `ask_concurrent` adds request identity/caller and an optional Thinker factory to a seat-managed ask. It is not automatic deployment of a distributed cluster. A returned `other_server`/error outcome is not an answer. Validate the chosen worker/storage layout under actual workload.

Installed commands: `cpu-llm-demo`, `cpu-llm-conversation-candidates`, `cpu-llm-serve`, and `cpu-llm-ingest`. The ingestion command operates on bundled corpora; use the Python ingestion API for custom documents. Help/invalid flags exit before model/database construction. External conversation candidate exports remain a review queue, not auto-imported runtime answers or new medical facts.

13 Evidence and reproducibility

The published 1.10.0 release passed strict distribution metadata checks. Archive verification checked 125 runtime/resource files, including the trained projection, data, demo resources and installed commands. A clean base installation with no system site packages passed dependency checks and both installed demos: healthcare 12/12 and conversation 12/12 authored checks. The final source test suite passed 897 tests, with 2 skipped and 3 pre-existing expected failures. The full suite and the clean-install demos are different validation scopes.

Historical authored comparisons provide additional engineering examples, not clinical or hosted-model superiority results. The 32-case medical run at revision `26a5215` reported full-contract passes of medication: ordinary 9/16, strict 16/16, simple literal/calculator 9/16; healthcare records: ordinary 15/16, strict 16/16, baseline 10/16. Correct visible arithmetic with a held final status was counted as a delivery-contract failure, not fabricated number drift. Some tasks require correct refusals rather than answers. The ordinary, strict and baseline paths are library/components; no generative LLM arm was installed or run in that comparison.

A separate 42-case authored strict evaluation reported 42/42 versus 25/42 for its limited literal/calculator baseline. The unchanged 24-case domain replay reported 24/24 after routing/precision/context fixes. These runs used earlier recorded revisions and different cases/scorers; they must not be added together or presented as a single 1.10.0 clinical accuracy rate. The data, frozen receipts, raw responses, corrected scorer history and hashes are retained in Git.

The 1.10.0 published upload artifacts were verified against the tested SHA-256 hashes. Neither distribution validation nor a perfect authored fixture score proves clinical safety, arbitrary question coverage, data quality, security certification, universal hallucination prevention or competitive LLM accuracy. No independent clinical gold/sign-off was supplied. No energy, carbon, latency or cost advantage over a hosted model was measured here.

14 Deployment checklist and next steps

Start with the installed fictional demos and a small representative corpus. Keep source normalization, temporal validity, caller scope and the selected response mode explicit. Retain accepted evidence and refusal reasons. Define what the application may do with quotations, calculations, partial results and policy replies before enabling autonomous actions.

For healthcare, establish authoritative source/version policy, clinical intended use, independent expert gold/review and separately validated treatment/diagnostic rules if those decisions are intended. For finance, validate source authority, numeric policy, precision, input completeness and action approval. The library alone does not provide those organizational or professional controls.

Use cached assets and restricted egress when offline operation is required. Benchmark startup, ingestion, memory and concurrency on the actual hardware; do not infer a performance SLA from a small demo. Re-run supported and refusal cases after changing parser assets, schemas, rules, source readers or dependency versions. Keep unsupported forms visible instead of silently widening claims.

The next step for a developer is concrete: install 1.10.0, run the two demos, inspect their JSON receipts, ingest a representative document and verify the request/result contract against the application's requirements.

References and implementation authority

1. Published package: https://pypi.org/project/cpu-llm/1.10.0/

2. Repository and license: https://github.com/insightitsGit/cpullm

3. Release source: https://github.com/insightitsGit/cpullm/tree/0102bc0

4. Release verification: https://github.com/insightitsGit/cpullm/blob/40ead1e/docs/releases/1.10.0-build-validation.md

5. Strict contract: https://github.com/insightitsGit/cpullm/blob/0102bc0/docs/strict-evidence-spec.md

6. Conversation contract: https://github.com/insightitsGit/cpullm/blob/0102bc0/docs/conversation-spec.md

7. Stylistic specification: https://github.com/insightitsGit/cpullm/blob/0102bc0/docs/stylistic-generation-spec.md

8. Flag registry: https://github.com/insightitsGit/cpullm/blob/0102bc0/src/cpu_llm/flag_contract.py

9. Python pipeline: https://github.com/insightitsGit/cpullm/blob/0102bc0/src/cpu_llm/pipeline.py

10. HTTP service: https://github.com/insightitsGit/cpullm/blob/0102bc0/src/cpu_llm/serve.py

11. Healthcare demo and actual replies: https://github.com/insightitsGit/cpullm/tree/0102bc0/examples/healthcare

12. Medical comparison: https://github.com/insightitsGit/cpullm/blob/0102bc0/docs/benchmarks/2026-11-02-medical-local/REPORT.md

13. Strict evidence acceptance: https://github.com/insightitsGit/cpullm/blob/0102bc0/docs/benchmarks/2026-11-02-evidence-v2/REPORT.md

14. Domain replay: https://github.com/insightitsGit/cpullm/blob/0102bc0/docs/benchmarks/2026-11-02-domain-replay-v2/REPORT.md

15. Healthcare acceptance gate: https://github.com/insightitsGit/cpullm/blob/0102bc0/docs/healthcare-evidence-acceptance.md

This is an implementation whitepaper with AI-assisted drafting and mechanical API/artifact checks. It is not an independent peer review. Example labels, records and dialogue facts are fictional unless explicitly identified otherwise.